

Содержание

Кратко

Массив — это структура, в которой можно хранить коллекции элементов — чисел, строк, других массивов и так далее. Элементы нумеруются и хранятся в том порядке, в котором их поместили в массив. Элементов может быть сколько угодно, они могут быть какими угодно.

Массивы очень похожи на нумерованные списки.

Как пишется

Создадим массив с помощью квадратных скобок `[]`.

К примеру, можно создать пустой массив:

```
1 const guestList = [] // гостей нет
```

Скопировать

А можно создать сразу с элементами внутри:

```
1 const theGirlList = [
2   'Серсея', 'Илин Пейн', 'Меррин Трант', 'Дансен', 'Гора'
3 ]
```

Скопировать

Элементы могут быть разных типов:

```
1 const infoArray = [
2   'Россия',
3   ['Москва', 'Санкт-Петербург', 'Казань', 'Екатеринбург'],
4   [true, true, false, true]
5 ]
```

Скопировать

Внутри массива могут быть другие массивы:

```
1 const arrayOfArrays = [
2   'Россия',
3   ['Москва', 'Санкт-Петербург', 'Казань', 'Екатеринбург'],
4   [true, true, false, true]
5 ]
```

Скопировать

Как понять

Массивы хранят элементы в пронумерованных «ячейках». Нумерация начинается с нуля. Первый элемент массива будет иметь номер 0, второй — 1 и так далее. Номера называют **индексами**.

Количество доступных ячеек называют *размером* или *длиной* массива. В JavaScript длина массива обычно совпадает с количеством элементов в нем. Массивы хранят свой размер в свойстве `length`:

```
1 const infoArray = [
2   'Россия', 'Москва', 144.5, 'Russian ruble', true
3 ]
4 console.log(infoArray.length)
5 // 5
```

Скопировать

Чтение

Чтобы получить содержимое ячейки с этим номером, обратитесь к конкретному индексу. Если ячейка пустая или такой ячейки нет, то JavaScript вернёт `undefined`:

```
1 const guestList = ['Маша', 'Леонард', 'Шелдон', 'Джон Сноу']
2 const firstGuest = guestList[0]
3
4 console.log(firstGuest)
5 // Маша
6
7 console.log(guestList[3])
8 // Джон Сноу
9
10 console.log(guestList[999])
11 // undefined
```

Скопировать

Запись

Используйте комбинацию чтения и оператора присваивания:

```
1 const episodesPerSeasons = [10, 10, 10, 10, 10, 9, 7, 6]
2
3 console.log(episodesPerSeasons[5])
4 // 9
5
6 // Запись в ячейку с индексом 5
7 episodesPerSeasons[5] = 10
8 console.log(episodesPerSeasons[5])
9 // 10
```

Скопировать

Добавление элементов

Добавление элементов — это частая операция. Для добавления используйте методы:

- `push()` — для добавления в конец массива.
- `unshift()` — для добавления в начало массива.

Оба принимают произвольное количество аргументов. Все аргументы будут добавлены в массив. Лучше использовать `push()`, он работает быстрее. Методы возвращают размер массива после вставки:

```
1 const watched = ['Властелин Колец', 'Гарри Поттер']
2
3 watched.push('Зелёная Книга')
4 console.log(watched)
5 // ['Властелин Колец', 'Гарри Поттер', 'Зелёная книга']
6
7 let newLength = watched.push('Мстители', 'Король Лев')
8 console.log(newLength)
9 // 5
10
11 newLength = watched.unshift('Грязные танцы')
12 console.log(newLength)
13 // 6
14
15 console.log(watched)
16 // ['Грязные танцы', 'Властелин Колец', 'Гарри Поттер',
17 //  'Зелёная книга', 'Мстители', 'Король Лев']
18 // ]
19
```

Скопировать

Создать большой массив из чисел

С помощью цикла и метода `push()` можно быстро создать большой массив с числами.

Создадим массив чисел от 1 до 1000:

```
1 const numbers = []
2 for (let i = 1; i <= 1000; ++i) {
3   numbers.push(i)
4 }
```

Скопировать

Создадим массив чётных чисел от 0 до 1000:

```
1 const evenNumbers = []
2 for (let i = 0; i <= 1000; i += 2) {
3   evenNumbers.push(i)
4 }
```

Скопировать

Поиск по массиву

Используйте `indexOf()`, чтобы найти, под каким индексом хранится элемент.

Используйте `includes()`, чтобы проверить, что элемент есть в массиве:

```
1 const episodesPerSeasons = [10, 10, 10, 10, 10, 9, 7, 6]
2
3 console.log(episodesPerSeasons.includes(8))
4 // false
5
6 console.log(episodesPerSeasons.includes(6))
7 // true
```

Скопировать

Интересно, что если в массиве будут индексы с пропусками, то можно получить разрежённый массив. Предположим, у нас есть набор элементов:

```
1 const arr = ['d', 'o', 'k', 'a']
```

Скопировать

Добавим к нему ещё один элемент, так, чтобы его индекс был больше длины всего набора элементов. Мы получим массив с незаполненным элементом (empty slot). Если обратимся к нему по индексу, получим `undefined`:

```
1 arr[5] = '!'
2
3 console.log(arr)
4 // Выведет в Firefox ['d', 'o', 'k', 'a', <1 empty slot>, '!']
5
6 console.log(arr[4])
7 // Выведет undefined
```

Скопировать

Длина массива будет включать в себя все элементы, включая незаполненные, то есть в нашем случае не 5 элементов, а 6:

```
1 console.log(arr.length)
2 // Выведет 6
```

Скопировать

Или мы можем взять другой пример:

```
1 // Зрители, которые заняли три места в ряду. Индексы их мест: 0, 1, 2
2 const audience = ['👤', '👤', '👤']
3
4 // Оползавший зритель занял место в темноте и не по порядку!
5 audience[5] = '👤'
6
7 // Места с индексами 3 и 4 всё ещё свободны, и это логично!
8 console.log(audience)
9 // Array(6) [ '👤', '👤', '👤', <2 empty slots>, '👤' ]
```

Скопировать

На практике

Николай Лопин советует

Копирование массива

С копированием есть проблема. Массив — большая структура, и она не вставляется в одну переменную. Переменная хранит адрес, где находится массив. Если этого не знать, то результат такого кода будет выглядеть странно:

```
1 const iWatched = ['GameOfThrones', 'Breaking Bad']
2 const vitalikWatched = iWatched
```

+ Развернуть `ed.push('American Gods')`

На собеседовании

Задать вопрос в рубрику

Есть коллекция произвольных объектов. Вам нужно выкинуть все не уникальные элементы. Порядок не важен.

Я знаю ответ

Viktar Nezhbart отвечает

Основной сложностью данной задачи является определение уникальности элементов коллекции.

Попробуем для начала упростить задачу. Допустим, элементами коллекции являются примитивные значения, а сама коллекция - это массив. В этом случае наше решение может быть таким:

```
1 function getUnique (array) {
2   // это не массив, возвращаем пустой массив
3   if (!Array.isArray(array) || array.length === 0) return []
4   // инициализируем объект для хранения уникальных элементов
5   const unique = {}
6   for (let i = 0; i < array.length; i++) {
7     unique[array[i]] = true
8   }
9   return Object.keys(unique)
10 }
```

+ Развернуть

Дан одномерный массив. Его элементами могут быть значения разных типов, включая `undefined`, `null`, `boolean`, `string`, `number`.

Например:

```
1 const array = [5, undefined, 0, false, '', null, true, 1]
```

Скопировать

Массив может быть разрежённым (sparse array), то есть включать незаполненные элементы (empty slots).

Например, добавим к массиву `array` элемент с индексом 15:

```
1 array[15] = 'новый элемент'
2 console.log(array)
3 // [ 5, undefined, 0, false, '', null, true, 1, <7 empty items>, 'новый элемент' ]
```

Скопировать

Напишите функцию подсчёта незаполненных элементов массива.

Я знаю ответ

Это вопрос без ответа. Вы можете помочь! Чтобы написать ответ, следуйте инструкции.

Читайте также

Хранение по ссылке и по значению

Брюки превращаются в элегантные шорты, а во что превращается строка «55ab» при преобразовании в число?

Почти всё в JavaScript — объект

Что бы мы ни использовали при написании JavaScript-кода, почти всё это объекты под капотом.

Авторы: Николай Лопин

Контрибьюторы: Егор Левченко, Анастасия Ярош

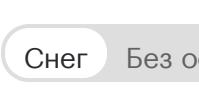
Редактировать на  GitHub

Обновлено 13 декабря 2024

Статья была полезной?



Если вы нашли ошибку, отправьте нам [пулреквест!](#)



Рассылка



[0](#) [проекте](#)

[Спецпроекты](#)

[Участники](#)

[Лицензии](#)

[Документация](#)

Тема:

